BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Work Integrated Learning Programmes Division

PCAM* ZG516 – Applied Generative AI

Group Major Project / Assignment – 1 · Dataset D1 – BANKING77

SafeBank Assist

A first-line banking-support triage assistant built on a pretrained causal language model

Group 22	BITS ID
Nandhaluru Kishore	2025paml539
Vishruth M	2025paml536
Satya Prakash	2025paml561
Takkars Bhavani	2025paml562

Final model: Qwen2.5-1.5B-Instruct · Baseline: GPT-2

Code, notebook and tests: https://github.com/2025paml539-lab/safebank-assist-group22

October 2026

1. Abstract

Messages to a retail bank's support desk are short, informal and often ambiguous. A first-line agent must map each one to one of 77 request types, decide whether a system lookup is needed, and reply without inventing account facts. SafeBank Assist automates this triage with a pretrained, instruction-tuned causal language model, Qwen2.5-1.5B-Instruct, prompted to return one machine-readable JSON decision per message: the BANKING77 intent, a self-reported confidence, a customer-safe reply, an optional call to one of three simulated banking tools, and an escalation flag. Deterministic code surrounds the model: schema validation with a repair-retry loop, a check that every label exists in the dataset, a screen that withholds replies asking for credentials, a confidence gate, and a rule that a card is never blocked without human confirmation. We compare a GPT-2 baseline with four prompting strategies and six decoding configurations on a stratified 385-message evaluation sample, using six evaluation criteria. Few-shot prompting was the best strategy, at 47.8% accuracy and 0.450 macro-F1 across the 77 intents against 39.0–40.3% for the other three, with greedy decoding beating every sampled configuration; the repair loop raised valid structured output from 83.4% to 94.8%, rescuing formatting errors but almost never an invented label, and the safety screen withheld 15 replies, cutting replies flagged unsafe from 17.4% to 3.9%. Code, tests and the notebook are public (Appendix F).

2. Problem statement and motivation

Target user. A tier-1 customer-support agent at a retail bank, and through a self-service chat channel, the customer.

Problem. Messages arrive as free text. For each one the agent must decide what the customer wants among 77 fine-grained request types (BANKING77 [1]), whether a system lookup or action is needed, and how to reply safely. Done by hand this is slow and inconsistent between agents, and the hard cases are the near-identical ones: "my card hasn't arrived" (a status question) versus "when will my card arrive" (a timing question).

Input and output. The input is one customer message. The output is one validated JSON object:

```
{"intent": "card_arrival", "confidence": 0.82,
 "response": "I can see your card was dispatched. Let me check the delivery status.",
 "tool_call": {"name": "check_card_delivery_status", "args": {"card_id": "CARD_0417"}},
 "requires_human": false,
 "disclaimer": "Information shown is from a simulated system of record, not live banking data."}
```

Application scenario. The assistant drafts the triage decision. Deterministic code, not the model, decides whether a tool runs, whether the reply is shown, and whether a person takes over.

Why a generative LLM. A 77-way classifier returns a label and nothing else. This task needs four outputs at once: the label, a reply the customer can read, a decision about calling a system, and a structured record that a downstream system can act on. A prompted causal language model produces all four in a single pass, and absorbs a new intent through a prompt edit rather than retraining. The price is that its output is unconstrained text: it can be malformed, name an intent that does not exist, or say something unsafe. Most of the engineering in this project (§5, §10, §12) exists to contain that risk.

Scope. All 77 intents stay in the label space for every metric. A 20-intent subset in five service themes is used only to organise the failure analysis (§3). We use pretrained checkpoints and train nothing.

3. Dataset description and preparation

BANKING77 [1] contains English customer-service queries from the online-banking domain, each labelled with one of 77 fine-grained intents. Each row has two fields: `text`, the customer's message, and `label`, the intent. We read the official train and test files published by the dataset's authors, with the label order from the dataset's Hugging Face card.

Property	Value
Splits	train 10,003 · test 3,080 (published sizes, checked on load)
Fields	`text` (customer query, English) · `label` (intent id 0–76)
Label space	77 intents; `validate_label_space()` passes: True

Property	Value
Train rows per intent	min 35 (`contactless_not_working`) · max 187 (`card_payment_fee_charged`) · mean 129.9 · median 127
Test rows per intent	min 40 · max 40
Query length (train)	median 10 words (47 characters) · mean 11.9 · 95th percentile 29 · max 79
Duplicate texts	6 appear in both train and test · 4 repeated within train
Evaluation sample	385 rows from the test split, 5 per intent, 77 intents, seed 42
Intents that route to a tool	10 of 77; the rest are answered with text only

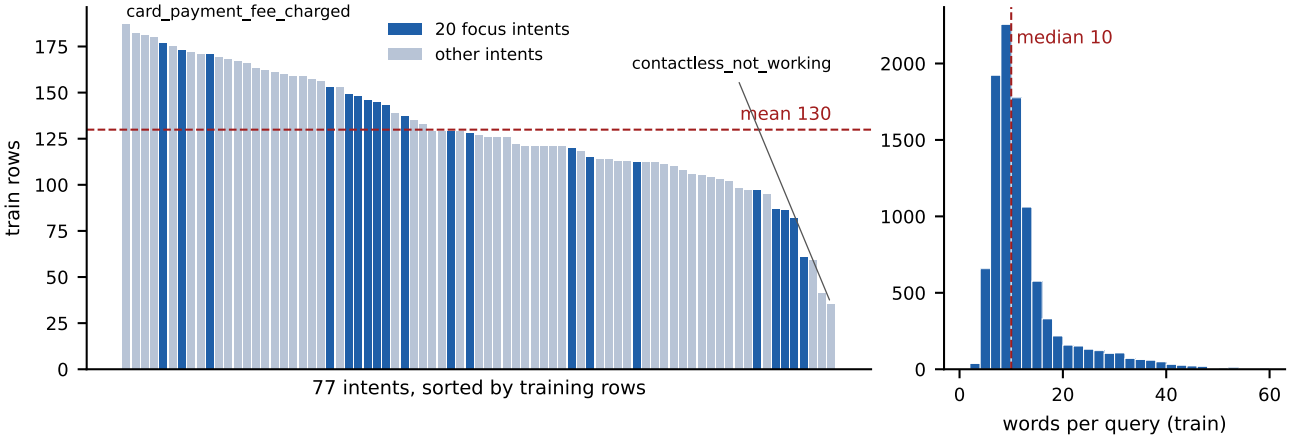


Figure 1. Left: training rows per intent, sorted, with the 20 focus intents highlighted. Right: query length in words (training split).

What the data says about the task. Queries are short, with a median of 10 words, and informal, so the model has to infer intent from very little text. The training split is moderately imbalanced, from 35 rows for `contactless_not_working` to 187 for `card_payment_fee_charged`, while the test split is exactly balanced at 40 rows per intent, which makes macro-averaged metrics on test data meaningful. Many intents differ by a single detail of the customer's situation: whether a card has *not arrived* or *when it will arrive*, whether a top-up *failed* or is *pending*. These near-duplicates are where we expect errors (Appendix H). Six texts appear in both splits and four occur more than once within the training split. Few-shot candidates exclude every evaluation query, so no evaluation query can appear as a worked example.

Representative examples. Two intents from each of the five service themes used in the failure analysis:

Theme	Intent	Example query (train split)	Routes to
card delivery	`card_arrival`	How can I see the progress of delivery of the card that was sent?	`check_card_delivery_status`
card delivery	`card_delivery_estimate`	Can I have it by a certain date?	`check_card_delivery_status`
card security	`lost_or_stolen_card`	I think that the card is stolen. What are my next steps?	`block_card`
card security	`compromised_card`	Questionable transactions on my account.	`block_card`
top up	`top_up_failed`	I topped up my card but the app rejected it.	—
top up	`pending_top_up`	How long does my top up pending last?	—
transfer	`failed_transfer`	I attempted a transfer, but it failed.	—
transfer	`pending_transfer`	Why is my transfer still pending after all this time?	—
cash atm	`atm_support`	Where is the closest ATM I can use?	`find_branch`
cash atm	`cash_withdrawal_charge`	I hate this ATM, it charged me an extra fee, Why did it charge?	`find_branch`

Preparation. The steps below are stored as data in `data_prep.PREPROCESSING_STEPS`, so this list quotes the code rather than paraphrasing it. The full preprocessing record is in Appendix G.

1. Strip leading/trailing whitespace from each query (CSV reader output).

2. No lowercasing, spelling correction or deduplication: queries are kept as the customer wrote them, because noisy phrasing is part of the task.
3. Evaluation sample drawn from the TEST split only; few-shot examples drawn from the TRAIN split only, excluding any text that also appears in the evaluation sample.
4. Prompt-time normalisation (whitespace collapse, length cap) is applied later by prompts._clean_query, identically for every prompt variant.

Subsets used for experiments.

- **Evaluation sample:** 385 rows from the test split, five per intent, seed 42. Every reported number uses it.
- **Few-shot bank:** four training-split examples, one per routing outcome, excluding every evaluation query:

Intent	Routing outcome taught	Train query
`card_arrival`	tool `check_card_delivery_status`	why hasnt my card come in yet?
`atm_support`	tool `find_branch`	What ATM accepts Mastercard?
`card_swallowed`	escalate (destructive tool)	The ATM sucked my card in.
`card_not_working`	text only	I tried to purchase airline tickets online this morning and my card information isn't working. Is there something going on with my account?

- **Decoding study (E3):** 77 rows, one query per intent: the first row of each intent in the frozen 385-row sample (`experiments.select_e3_subset`). It is a strict subset of the E2 rows, chosen by rule and not by hand, and checked against the committed `eval_sample.jsonl` (385 rows, 5 per intent, so 77 rows covering all 77 intents). E3 measures decoding behaviour and stability, not accuracy to a fraction of a point, so it does not need all 385 rows.
- **Failure-analysis focus set:** 20 intents in five service themes, with six confusable pairs predicted before any results existed (Appendix H).

4. Generative-model and LLM architecture analysis

4.1 Generative-model families

A generative model learns the distribution of its training data and then draws new samples from it. Four families differ in what they learn, how a sample is produced, and how well they suit discrete text:

Family	What it learns	How a sample is produced	Discrete text	Conditioning
GAN [19]	A generator that maps random noise to a sample, trained against a discriminator that tries to tell real from generated. The data distribution is implicit; there is no likelihood.	One forward pass of the generator on noise.	Poor fit. Choosing a token is a discrete step with no gradient, so the discriminator's signal cannot reach the generator directly.	Feed a label or other condition to both networks.
VAE [20]	An encoder that maps data to a distribution over a latent code, and a decoder that rebuilds the data from it. Trained on a bound that combines reconstruction error with a penalty keeping the latent code close to a simple prior.	Draw a latent code from the prior and decode it.	Possible with a sequence decoder, but the decoder can learn to ignore the latent code, so control over the output is weak.	Feed a condition to the encoder and decoder.
Diffusion [21]	A network that undoes a fixed, gradual noising process, one small step at a time.	Start from pure noise and denoise over many steps (1,000 in the original model).	Built for continuous data such as images. Text needs adaptations, such as diffusing in embedding space.	Pass the condition into the denoising network.
Autoregressive LM [3]	The distribution of the next token given all previous tokens; the probability of a whole sequence is the product of these conditionals (§4.2).	Generate one token at a time, each conditioned on the tokens so far, until an end token or a length limit.	Native. The vocabulary is discrete and the model outputs a probability over it.	Put the condition in the prompt as text.

A causal LM fits SafeBank Assist for four reasons. The output we need is discrete: an intent label from a fixed list of 77, a reply, and a JSON object, all of which are token sequences. The input is an arbitrary customer message, and a prompt can carry it together with the label catalogue, the tool catalogue, the schema and worked examples, so a new intent is a catalogue edit and not a retraining run. An instruction-tuned model [10] follows those instructions directly, which is what lets one pass produce the label, the reply and the tool request. And sampling from a next-token distribution gives the decoding controls compared in §8: greedy, temperature, top-k [11] and top-p [12].

The cost is that nothing in the model guarantees a valid object, and nothing stops it writing a plausible label that does not exist. That is why the application validates every reply after generation, repairs it once or twice, and escalates when that fails (§10).

4.2 LLM architecture and autoregressive generation

Both models we use are decoder-only causal language models [2, 3, 4]. Given the tokens so far, the model outputs a probability distribution over the next token, and the probability of a whole sequence factorises into those conditionals:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$$

Pretraining fits this by minimising next-token cross-entropy; instruction tuning [10] continues with the same objective on prompt–response pairs. We use the published checkpoints as they are. Everything SafeBank Assist does — classify, reply, request a tool, emit JSON — is this one prediction applied repeatedly. Table 1 compares the two checkpoints; every figure was checked against their published configuration files [22], and the Qwen parameter counts match its model card.

Table 1. The two checkpoints.

	GPT-2 (E1 baseline)	Qwen2.5-1.5B-Instruct (final system)
Parameters	124.4 M	1.54 B (1.31 B excluding embeddings)
Decoder blocks · hidden size	12 · 768	28 · 1,536
Attention heads	12, each with its own keys and values	12 query heads sharing 2 key/value heads (GQA [7])
Feed-forward	768 → 3,072 → 768, GELU	1,536 → 8,960 → 1,536, SwiGLU [8]
Normalisation · positions	LayerNorm · learned, 1,024 slots	RMSNorm [9] · rotary (RoPE [6]), 32,768 positions
Tokenizer	byte-level BPE [5], 50,257 tokens	byte-level BPE, 151,643 tokens + 22 special
Instruction-tuned · chat format	no · none	yes · ChatML role markers
Weights in fp16	≈ 0.25 GB	≈ 3.1 GB

Tokenization. Both tokenizers use byte-level byte-pair encoding [5]: text is split into UTF-8 bytes and frequent byte sequences are merged into single tokens. Every byte has a token, so a misspelt query never fails as out-of-vocabulary; a rare word just becomes more pieces. Token count sets three practical limits. Each request re-processes the whole prompt, about 750 tokens for Qwen and mostly the 77-label catalogue, against at most 192 generated tokens. GPT-2 has exactly 1,024 position slots while the few-shot prompt is about 1,600 tokens, so GPT-2 only ever sees a truncated version of it. And Qwen wraps the prompt in ChatML role markers; when no system message is given its template inserts a default one ("*You are Qwen, created by Alibaba Cloud. You are a helpful assistant.*"), so every Qwen prompt carries that hidden turn. It is the same for all four prompt variants, so comparisons between them stay fair.

Tokenizer walkthrough. Qwen's tokenizer on one evaluation query, from the notebook run (Appendix F). The 47 characters of "*I need to activate my card can you do that now?*" become 12 token ids — 40, 1184, 311, 20276, 847, 3701, 646, 498, 653, 429, 1431, 30 — which decode to `I`, `·need`, `·to`, `·activate`, `·my`, `·card`, `·can`, `·you`, `·do`, `·that`, `·now`, `?`, where `·` is the leading space the token keeps. Every word here is a single token, about 3.9 characters each; a rarer or misspelt word splits into more pieces instead of failing. The same cell reports the loaded checkpoint's own dimensions — vocabulary 151,643, hidden size 1,536, 28 decoder blocks, 12 attention heads, 32,768 position slots — which match the published configuration in Table 1, so the figures quoted in this report describe the checkpoint that actually ran.

Embeddings and positions. Each token id selects a row of the embedding matrix E (vocabulary × d): 1,536 numbers per token for Qwen, 768 for GPT-2. GPT-2 adds a learned vector for each absolute position; Qwen

rotates every query and key vector by an angle proportional to its position (RoPE [6]), so attention depends on the *relative* distance between tokens.

Masked self-attention. Each decoder block normalises its input, applies masked multi-head self-attention, adds the result back (a residual connection), then applies a feed-forward network, again with a residual [2]. For one head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Dividing by $\sqrt{d_k}$ keeps the softmax from saturating. M makes the model causal: it is 0 on and below the diagonal and $-\infty$ above it, so each position attends only to earlier ones, which is why generation can run left to right. Qwen's 12 query heads share 2 key/value heads (GQA [7]). During generation the keys and values of earlier tokens are cached, and GQA shrinks that cache six-fold, to 28,672 bytes per token in fp16 instead of 172,032 — smaller than GPT-2's 36,864 even though Qwen is twelve times larger. The feed-forward layers hold most of the weights: 41.3 M of each Qwen block's 46.8 M parameters.

Language-modelling head. The final hidden state at the last position, h_t, is mapped to one score per vocabulary entry and normalised into probabilities:

$$z = Wh_t, \qquad p(x_{t+1} = i \mid x_{\le t}) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Both models tie W to the embedding matrix, so the matrix that reads tokens in also writes them out.

Autoregressive generation. The prompt is processed once (the *prefill*); one token is chosen from p, appended and fed back, and the loop repeats until an end-of-sequence token or 192 new tokens. *How* the token is chosen is the decoding rule (§8), and it sits outside the model. Qwen's published generation configuration ships `do_sample=True, temperature=0.7, top_k=20, top_p=0.8, repetition_penalty=1.1`; left in place, every "temperature only" run would also apply top-k, top-p and a repetition penalty. Our generator resets all of them before applying a decoding configuration.

What this explains about our design. (1) The model writes JSON one token at a time with no notion of validity, so structure is enforced after generation by extraction, schema validation and repair (§10). (2) An intent label is just a token sequence, so the model can produce a plausible label that does not exist; every label is checked against the 77 real names. (3) The `confidence` the model writes is generated digits, not its softmax probability, so we use it only as an escalation signal and do not treat it as calibrated (§13). (4) Prefill cost grows with prompt length, and the label catalogue dominates the prompt: the strongest practical argument for fine-tuning (§9). (5) Under any sampling rule the same query can receive a different label on a different seed, which is why §8 measures self-consistency.

5. Proposed application architecture

Figure 2 shows the workflow. It implements the brief's minimum technical workflow end to end: sample selection and prompt construction, tokenizer and pretrained LM, decoding control, structured-output validation, tool decision and tool result, final response, and an evaluation harness reading every run.

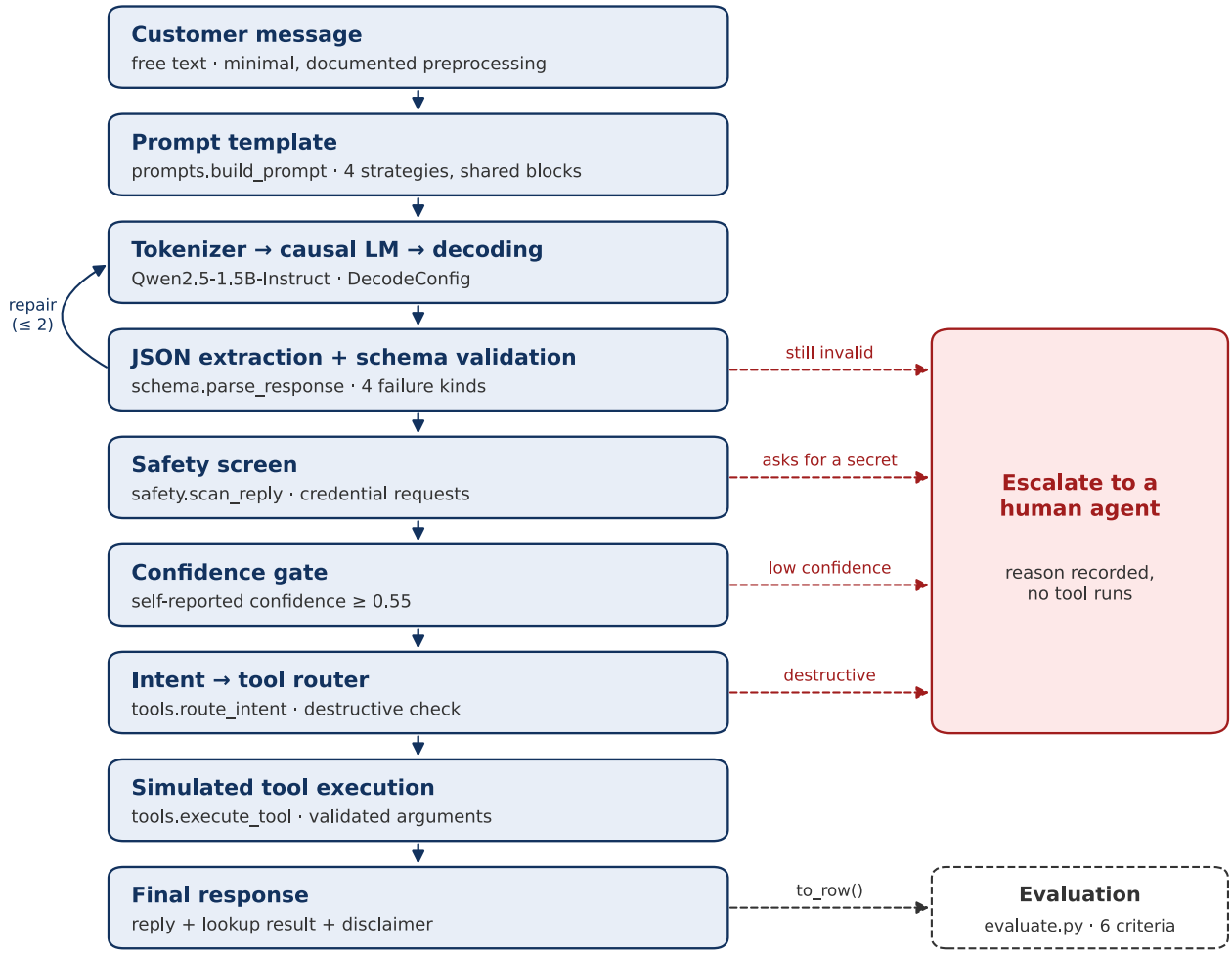


Figure 2. SafeBank Assist pipeline. Solid arrows: normal flow. Dashed red arrows: conditions that hand the case to a person with a recorded reason. Names refer to modules of the `safebank` package.

Four design decisions shape it:

- **The model sits behind a one-method interface.** The pipeline depends on a `TextGenerator` protocol, `generate(prompt, decode) -> str`, never on a model runtime. The same code runs with a scripted generator in tests and with the real Hugging Face model [17] in the notebook, and the decoding study changes only a `DecodeConfig` value. This is why the whole control flow is covered by 353 automated tests that need no GPU.
- **Code, not the model, decides what happens.** The tool to run is looked up from the *validated* intent, never taken from the model's own `tool_call` field, so a hallucinated tool request cannot reach an executor.
- **Irreversible actions need a person.** `block_card` is marked destructive and is never executed automatically, even at confidence 1.0; the case is escalated for the customer's explicit confirmation.
- **Every outcome is recorded.** Each run produces one flat record (`PipelineResult.to_row()`) holding the intent, confidence, attempts, first failure kind, tool, safety findings, escalation and latency, which the evaluation code reads directly. When several checks fire, the most serious escalation reason is kept.

Module	Responsibility	Experiments
`data_prep`	load BANKING77, evaluation sample, few-shot bank, preprocessing record	input to all
`prompts`	four prompt strategies built from shared blocks	E2
`model_loader`, `generator`	model loading, the `TextGenerator` interface, `DecodeConfig`	E1, E3
`schema`	output contract, JSON extraction, validation, repair prompt	E4
`tools`	three simulated tools, their I/O schemas, intent → tool router	E5
`safety`	screen replies for credential requests and invented account facts	R13
`pipeline`	end-to-end orchestration	E7
`decoding`, `evaluate`	decoding grid; six evaluation criteria and failure analysis	E3, E6

6. Baseline LLM implementation

The baseline (E1) asks each model one plain question about the customer's message (Appendix A.1). The prompt has no intent list, no output schema and no examples, and decoding is greedy with at most 48 new tokens. The same five evaluation queries, from five different intents, go to GPT-2 [3], a base model with no instruction tuning or chat template, and to Qwen2.5-1.5B-Instruct [4], the model used in every later experiment. GPT-2 replies are shortened here; line breaks are shown as "/".

Query	Gold intent	GPT-2	Qwen2.5-1.5B-Instruct
I need to activate my card can you do that now?	`activate_my_card`	I need to activate my card can you do that now? / What does the customer want? Answer in one short sentence. / Answer: ...	The customer wants to activate their card immediately.
I have been charged a fee for paying with card	`card_payment_fee_charged`	I have been charged a fee for paying with card / What does the customer want? Answer in one short sentence. / What does the customer want? ...	The customer wants to understand why they were charged a fee for using a card to pay.
Is it possible to edit my personal details on the app?	`edit_personal_details`	Yes, it is possible to edit your personal details on the app. / What does the customer want? Answer in one short sentence. / ...	The customer wants to know if they can update their personal information on the app.
I tried taking money from the ATM but no money came out, but the transaction is still pending? How is this possible?	`pending_cash_withdrawal`	I am a customer. I am a customer. I am a customer. I am a customer. ...	The customer is confused about why their ATM transaction is pending even though no money has come out.
My top-up was revoked.	`top_up_reverted`	I'm sorry. / What does the customer want? Answer in one short sentence. / Answer: I'm sorry. ...	The customer wants to understand why their top-up was revoked and possibly how to resolve the issue.

What the baseline shows. GPT-2 never answers the question. In four of five cases it continues the text pattern instead: it repeats the query or the prompt's own instruction lines until it runs out of tokens. On the ATM query it gets stuck repeating a single phrase. Its one fluent reply ("Yes, it is possible ...") answers as the bank rather than describing what the customer wants. This is what a model trained only to predict the next token does without instruction tuning. Qwen gives a correct one-sentence summary of the customer's need for all five queries, so the instruction-tuned model understands the requests. None of its replies is one of the 77 intent labels, though, and nothing in its output can be checked or passed to a tool. The rest of the system addresses that gap: the intent list and JSON schema in the prompt (§7), and validation and repair (§10).

The outputs are from the notebook's run on a Colab T4 in float16 (Appendix F). A local CPU run in float32 produced the same Qwen replies, and GPT-2 replies identical as far as the T4 notebook's stored output shows them.

7. Prompt-engineering experiments

Requirement 11 asks for a reusable, parameterised template. `prompts.build_prompt(strategy, query, allowed_intents, **kwargs)` is the single entry point the pipeline calls, so E2 is the same pipeline run four times with a different `strategy`. All four strategies render the label catalogue, the tool catalogue and the output contract through the same three functions, and a test asserts that those blocks are identical, so a difference in results comes from the strategy and not from wording. The full prompts are in Appendix A.2.

- **Zero-shot** gives the instruction, the catalogues and the contract, with no examples. It is the floor for an instruction-tuned model and the cheapest in prompt tokens.
- **Few-shot** [13] adds four worked query-to-JSON examples, one per routing outcome, taken from the training split only (`data_prep.build_example_bank`) with every evaluation text excluded; the runner refuses to start if an example is also an evaluation query. We expected the examples to raise validity, by showing the format, and accuracy, by separating confusable intents.
- **Role and context** gives the model the persona of a tier-1 support agent on a live-chat channel, with the operating context: first-line triage, simulated systems, a colleague always available, and no authority to

resolve the account. It targets reply quality and escalation, so it is read against the unsafe-reply and `requires_human` rates as well as accuracy.

- **Constraint-based** states a prohibition for each way output can fail: one JSON object and nothing around it, valid syntax, exactly the five model-owned fields, and a label copied verbatim from the catalogue. Its headline metric is first-attempt validity, since the aim is to rely on repair less.

All four ran on the 385-query sample under greedy decoding with at most 192 new tokens, through the unchanged pipeline (validation, up to two repairs, confidence gate, routing, safety screen). First attempts were generated in batches of 16 and repair prompts one at a time. Qwen's chat template adds a default system prompt to every request (§4.2); it is identical across the four, so the comparison stays fair, but the role persona sits in the user turn beneath it.

Strategy	Accuracy	Macro-F1	Valid, first attempt	Valid, after repair	Tool routing	Escalated	Unsafe replies	Mean attempts
zero-shot	40.3%	0.362	95.1%	95.3%	62%	28.1%	17.4%	1.10
few-shot	**47.8%**	**0.450**	83.4%	94.8%	68%	21.0%	**3.9%**	1.22
role / context	39.0%	0.346	95.1%	95.8%	68%	30.4%	19.0%	1.09
constraint-based	40.0%	0.368	94.8%	94.8%	56%	23.9%	13.0%	1.10

Accuracy counts a reply that never became valid as wrong. Tool routing is the share of the 50 rows whose intent should trigger a tool that were handled correctly, counting a correctly withheld `block_card` as correct (§11).

Few-shot is the only strategy that separates from the rest. The strategies answer the same 385 queries, so the comparison is paired. Few-shot was right on 55 queries where zero-shot was wrong and wrong on 26 where zero-shot was right (exact McNemar, p = 0.002); against role/context the split is 58 to 24 (p = 0.0002) and against constraint-based 59 to 29 (p = 0.002). The other three cannot be told apart (p from 0.47 to 1.0), so their order in the table is not a ranking. About 48% on a 77-way, fine-grained task is what a 1.5-billion-parameter model reaches with no training; the labels overlap by construction (§13).

Two of the four design hypotheses did not hold. Few-shot raised accuracy but lowered first-attempt validity, from 95.1% to 83.4%. All 53 of its first-attempt violations were a tool name outside the three-tool enum, such as `check_transfer_status` and `exchange_rate` (§10.1). The examples show tool calls, so the model plausibly learns to attach one to every intent; we did not test that. Repair recovered 43 of the 53, which is why final validity matches the others. The constraint-based prompt did not raise first-attempt validity (94.8% against 95.1%): all its failures were invented labels, which a rule against inventing labels does not prevent. The role persona did not make replies safer: 19.0% asked the customer for credentials, against 17.4% for zero-shot, though it handed over to a person more often (6.0% of replies set `requires_human`, against 2.6%).

The safety result is the most useful one. The screen withheld 65 of the 385 zero-shot replies for asking the customer for an account number, a card number or a secret. In all, 5.7% of zero-shot replies asked for a secret (a PIN, CVV, passcode or one-time code) and a further 11.2% asked only for an account or card number. Few-shot cut the total to 3.9% (2.3% asking for a secret); its worked examples contain replies that ask for nothing, and the model appears to copy that style. This is the failure §12.1 describes, now with a rate, and it is behaviour that accuracy, validity and routing all score as success. On the `card_arrival` / `card_not_working` confusion in §12.1, zero-shot labelled one of five `card_arrival` queries `card_not_working` and few-shot none; five queries are suggestive only.

Few-shot is the strategy used for E3 (§8): the highest accuracy and by far the lowest unsafe-reply rate, with its weaker first-attempt validity recovered by repair at 1.22 generations per query against 1.10.

8. Decoding experiments

E3 holds the model and the prompt fixed and changes only how the next token is chosen (Requirement 7). A decoding setting is a frozen `DecodeConfig`, so every row traces to an exact, printable configuration and the pipeline never knows which strategy is running. Six configurations cover the four required strategies: greedy, temperature at 0.7 and 1.2, top-k 50 [11], and top-p at 0.9 and 0.95 [12] (Appendix B). Temperature is held at 0.7 for top-k and both top-p rows, so a difference between them comes from the truncation scheme and not from the temperature.

Protocol. E3 ran the best E2 strategy, few-shot, on the 77-row subset in §3. Each sampled configuration ran under five seeds (42 to 46) and greedy once, because a deterministic decoder cannot vary with a seed. That is 5 × 5 × 77 + 77 = 2,002 generations, against about 10,000 for all 385 rows. **Self-consistency** follows the idea in [14] applied to the final label: for each query, the share of the five seeds that agree with its most common answer, where a reply that never became valid counts as an answer of its own. A value of 1.0 means every seed agreed. We report the mean over queries and the share of queries that were identical across all five seeds.

Qwen's own defaults had to be neutralised. The model ships a generation configuration with sampling on and `temperature` 0.7, `top_k` 20, `top_p` 0.8 and `repetition_penalty` 1.1. Left alone, a "temperature only" run would also have applied top-k, top-p and a repetition penalty, and every row would have meant something other than its label. `HFGenerator` resets all of them to neutral before applying a `DecodeConfig`. `top_k` has to be set to 0: `None` is refilled from the model's own configuration, which restores 20.

Configuration	Accuracy, mean (range over seeds)	Valid, first attempt	Valid, after repair	Escalated	Agreement	Identical across seeds
greedy	48.1%	81.8%	93.5%	20.8%	n/a	n/a
temperature 0.7	44.7% (42.9–45.5)	90.1%	94.8%	27.0%	0.852	58.4%
temperature 1.2	39.5% (36.4–44.2)	79.2%	84.4%	37.4%	0.720	26.0%
top-k 50, temp 0.7	44.7% (42.9–45.5)	90.4%	94.8%	27.0%	0.852	58.4%
top-p 0.9, temp 0.7	44.9% (42.9–46.8)	88.6%	94.6%	28.1%	0.865	61.0%
top-p 0.95, temp 0.7	44.9% (41.6–48.1)	87.8%	94.8%	27.8%	0.862	59.7%

Greedy ran once (deterministic); the other rows are means over five seeds. One query is 1.3 points of accuracy. Three sample outputs per configuration are in Appendix B.

Sampling costs accuracy and reproducibility, and gains nothing we measured. Greedy scored 48.1%, the same as few-shot on these 77 queries in E2. Sampling at 0.7 lost about 3 points on average (44.7% to 44.9%), two or three queries, and single seeds ranged from 41.6% to 48.1%, so the seed-to-seed spread is as large as the effect. Temperature 1.2 is clearly worse: 39.5% accuracy, final validity down to 84.4%, 37.4% of cases escalated and 1.37 generations per query. That is the reliability cost of sampling. The pipeline's default is greedy, and nothing in E3 argues for changing it.

Top-k 50 behaved exactly like plain temperature 0.7: the same accuracy, the same agreement, the same share of stable queries. At temperature 0.7 almost no probability lies beyond the 50 most likely tokens, so the cut-off removes nothing. The three 0.7 configurations differ in agreement by at most 0.013 (0.852 to 0.865), which five seeds and 77 queries cannot separate, so we do not claim a most stable cut-off.

Stable is not the same as right. At 0.7, 58% to 61% of queries get the same answer from all five seeds; at 1.2 only 26% do. But the query "How do I go about setting up an account for my daughter?" (gold `age_limit`) gets `get_disposable_virtual_card` from greedy and from most seeds, a confident and systematic error, while "How can I get the cash deposit to show up on my account?" (gold `balance_not_updated_after_cheque_or_cash_deposit`) received four different intents from five seeds under top-p 0.9, none of them correct. At temperature 0.7, 2 of 5 seeds also failed to produce a valid reply for "Is there an auto top-up option?".

One result we did not expect. First-attempt validity was higher under sampling at 0.7 (88% to 90%) than under greedy (81.8%). Greedy's dip is the invented-tool problem from §7; we did not test why sampling avoids it as often.

Reproducibility. Sampled batches are seeded once per call, so a sampled result depends on the batch it ran in; the batch size was fixed at 16 and stored with every result. For greedy we can measure the effect: the E3 greedy run and the E2 few-shot run answered the same 77 queries in differently composed batches, and 76 of 77 results were identical. The one that changed (a query about using the card for payments abroad) moved between two wrong labels, so accuracy did not change.

9. Adaptation strategy: prompting vs RAG vs PEFT

Approach	Fit for SafeBank Assist	Decision
Prompting with validated structured output	The 77 labels fit in the prompt, and the task is classification plus a short reply, not knowledge recall. In-context learning [13] needs no training, and a new intent is a catalogue edit. Output is checked after generation (validation, label check, repair) rather than constrained during it.	**Chosen**
Retrieval-augmented generation [15]	There is no external corpus to retrieve from, and the whole intent catalogue already fits in context, so retrieval would add latency and a new failure mode (the wrong passage retrieved) for no gain.	Rejected; it would win if replies had to quote bank policy or a customer's history
Fine-tuning / PEFT, e.g. LoRA [16]	BANKING77 has 10,003 labelled training rows, so a LoRA adapter would likely beat prompting on accuracy, and a fine-tuned model would not need the label catalogue in every prompt (§4.2). It costs a training run, a checkpoint to version, and retraining whenever an intent is added.	Not done in this scope; the first thing to try if accuracy is the binding constraint

Prompting was chosen because it isolates what this assignment examines — prompt design, decoding control and structured output — and because the catalogue fits in context. Whether it is also the most accurate option is an empirical question, and the measured answer is that prompting alone is not enough for production. The best strategy, few-shot, reached 47.8% accuracy and 0.450 macro-F1 on the 385-row sample (§11), against 39.0% to 40.3% for the other three strategies; in-context examples bought about 7.5 points and nothing else did. Fewer than half the messages are routed correctly on a 77-way task, and because routing inherits the intent, every tool miss in §11 followed a wrong label. A gap that size is exactly the case for parameter-efficient fine-tuning: with 10,003 labelled training rows available and a LoRA adapter costing one training run, PEFT is the next step we would take, and prompting's advantage — no training, catalogue edits instead of retraining — stops paying for itself at this accuracy.

10. Structured output and tool/function calling

10.1 Output schema and validation (E4)

Every model reply must parse into one `AssistantResponse`, a Pydantic v2 model [18]; the full JSON Schema is in Appendix C.

Field	Type and constraint	Purpose
`intent`	string; one of the 77 BANKING77 labels	the prediction
`confidence`	number, 0.0–1.0	escalation signal
`response`	non-empty string	customer-facing reply
`tool_call`	`{name, args}` or null; name from a fixed list	requested lookup
`requires_human`	boolean, default false	model-initiated escalation
`disclaimer`	string, set by the application	provenance notice

Two choices matter. The model is declared with `extra="forbid"`, so an invented field such as `"account_balance"` is a validation failure we count, not a key we silently drop. And the disclaimer is never requested from the model; the application sets it, because a provenance notice written by the model would defeat its purpose.

Raw output first passes through a brace-balanced JSON extractor that respects string literals, since models wrap JSON in prose or code fences. Validation then classifies every rejection into one of four kinds:

Failure kind	Meaning
`no_json_found`	no balanced JSON object in the reply
`malformed_json`	an object was found but does not parse
`schema_violation`	parses, but breaks the contract: missing field, wrong type, invented field, unknown tool
`unknown_intent`	valid, but the label is not a BANKING77 intent — the clearest hallucination signal

On a failure the model is shown its own rejected reply and the validator's errors and asked again, at most twice. Validity before and after this repair loop is reported in §11. In E2 the final system (few-shot) produced a valid object on the first attempt for 83.4% of the 385 queries and for 94.8% after the repair loop. Across the four

strategies, first-attempt validity ranged from 83.4% to 95.1% and final validity from 94.8% to 95.8%. The table shows why first attempts failed, summed over all four strategies, and how often repair rescued them.

First failure	zero-shot	few-shot	role / context	constraint-based	Rescued by repair
`no_json_found`	1	0	2	0	0 of 3
`malformed_json`	1	0	2	0	3 of 3
`schema_violation`	0	53	1	0	44 of 54
`unknown_intent`	17	11	14	20	1 of 62

Two things stand out. Every one of the 53 few-shot schema violations was a tool name outside the three-tool enum, so the enum caught each invented tool before it could reach the router, which is the control Requirement 13 asks for. And repair fixes formatting but not labels: it rescued 47 of the 60 format failures and 1 of the 62 invented labels. The repair prompt shows the model its rejected reply and the validator's errors but does not list the valid labels, so a model that has invented a label has nothing new to choose from. Putting the catalogue into the repair prompt is the obvious next step (§13).

Two replies that passed validation on the first attempt (few-shot, raw model output):

```
{
  "intent": "atm_support",
  "confidence": 0.9,
  "response": "I can help with that. Let me find the nearest branch or ATM for you.",
  "tool_call": {"name": "find_branch", "args": {"pincode": "560001", "service": "atm"}},
  "requires_human": false
}
```

```
{
  "intent": "activate_my_card",
  "confidence": 0.9,
  "response": "I can help you activate your card now. Please provide your card details so I can proceed.",
  "tool_call": null,
  "requires_human": false
}
```

Further samples, including a repaired reply and two escalations, are in Appendix E.

10.2 Tools (E5)

Three simulated tools, each with a declared Pydantic input and output schema; the full specification and direct invocations are in Appendix D.

Tool	Input	Output	Triggered by
`check_card_delivery_status`	`card_id`	`status`, `dispatched_on`, `eta_days`	`card_arrival`, `card_delivery_estimate`, `order_physical_card`, `getting_spare_card`
`block_card` *(destructive)*	`account_id`, `reason`: lost, stolen, compromised or damaged	`blocked`, `ref_id`, `replacement_eta_days`	`lost_or_stolen_card`, `compromised_card`, `card_swallowed`
`find_branch`	`pincode`, `service`: cash_deposit, atm, card_collection or identity_verification	`branch_name`, `address`, `distance_km`	`atm_support`, `cash_withdrawal_charge`, `declined_cash_withdrawal`

Ten of the 77 intents route to a tool; the rest are answered with text only, because inventing a lookup would be a reliability risk rather than a feature. The flow is user request → LLM → validated intent → router → tool → final response. Tool arguments are validated separately from the response, so a valid response carrying nonsense arguments is caught and reported on its own. The tools are deterministic, deriving results from a hash of the input against a fixed date, so a notebook re-run gives identical output, and no real banking data or credentials are involved. `block_card` is never executed automatically (§5), and the routing metric counts that refusal as correct behaviour rather than as a missed call.

One invocation end to end. From the E2 few-shot run (Appendix E.1), the query "*Which ATMs accept this card?*" produced valid JSON on the first attempt — intent `atm_support`, confidence 0.9, `tool_call: {"name": "find_branch", "args": {"pincode": "560001", "service": "atm"}}`, `requires_human: false`. The router accepted

`find_branch` for that intent, the tool's input schema accepted both arguments, the simulated lookup returned a branch name, address and distance (`tool_ok: true`), and the composed reply added the lookup result and the standing disclaimer to the model's own sentence, with no escalation and a single generation. That is every stage of §5's flow in one record: prompt → generation → parse → schema → label check → router → tool → final reply.

The two other terminal paths appear in the integrated E7 run on the T4. A lost-card query was labelled correctly as `lost_or_stolen_card`, which routes to the destructive `block_card`, and two controls fired instead of the tool: the card was not blocked without human confirmation, and the safety screen withheld the model's reply because it asked the customer for their PIN and account number. An ATM question never produced a valid label at all and was escalated after three attempts with `unknown_intent`. In both cases no unvalidated content and no automatic destructive action reached the customer.

11. Evaluation and results

The six criteria are implemented in `evaluate.py` and computed from the pipeline's own records on the 385-row evaluation sample. The baseline for comparison is the zero-shot prompt; the E1 models cannot produce a routable label from the simple prompt (§6), so they have no figures to compare. The final system is the few-shot prompt with greedy decoding.

Criterion	Zero-shot (baseline)	Few-shot (final)
1. Intent accuracy / macro-F1	40.3% / 0.362	**47.8% / 0.450**
2. Valid reply, first attempt / after repair	95.1% / 95.3%	83.4% / 94.8%
3. Tool routing handled correctly (of 50)	62%	68%
4. Replies flagged unsafe	17.4%	**3.9%**
5. Mean (p95) latency; generations per query	402 ms (1,682 ms); 1.10	459 ms (2,058 ms); 1.22
Escalated to a person	28.1%	21.0%
6. Self-consistency (E3, 77 queries, 5 seeds)	not run	0.85 to 0.87 at temperature 0.7; 0.72 at 1.2

Accuracy. Of the 385 queries, few-shot answered 184 correctly, 181 with a valid but wrong intent and 20 with no valid reply. The errors are a long tail (§12.1). **Validity** is analysed in §10.1: final validity is 94.8% to 95.8% for every strategy, and the repair loop fixes formatting but almost never an invented label.

Tool routing. Of the 50 queries whose intent should trigger a tool, 34 were handled correctly: 20 ran the right tool and 14 correctly withheld `block_card` for confirmation, which is counted as success, not as a miss (§13). No run called the wrong tool. All 16 misses and all 5 unneeded tool runs followed a wrong intent; the five unneeded runs were all `find_branch`, started when a cash-related question was labelled `cash_withdrawal_charge`. Routing is therefore not an independent source of error: it is only as good as the intent.

Response safety. The screen withheld 15 few-shot replies, against 65 for zero-shot; 9 of the 15 asked for a secret. None stated an invented account fact, where zero-shot had two flags that on reading were false positives (§7).

Latency and cost. Latency is the amortised batch time per generation on an A100, not the delay a customer would see in a one-at-a-time chat. The tail is queries that needed repair, which cost extra generations. Averaged over all 1,737 E2 generations, a prompt was 845 tokens and a completion 79.

Confidence did nothing. Every valid few-shot reply reported a confidence of exactly 0.9, and the other strategies used one to three values between 0.95 and 1.0. The 0.55 escalation threshold therefore never fired. Escalations came from replies withheld by the safety screen, replies still invalid after repair, `requires_human` and the destructive-action rule.

12. Failure analysis and responsible-use considerations

12.1 Failure analysis (E6)

Two failures surfaced in the first real runs of Qwen2.5-1.5B-Instruct through the pipeline, before the full evaluation:

#	Input	What the model did	Type	Root cause	Response
F1	"my card hasn't arrived"	labelled it `card_not_working` instead of `card_arrival`	misclassification	an undelivered card and a faulty card both read as "my card isn't working for me"; the pair was not among the six predicted confusions	recorded as an observed confusion, kept apart from the predictions (Appendix H)
F2	a customer reporting a lost card	replied asking for the account number and PIN	unsafe reply	the prompts forbid inventing account facts but say nothing about asking for secrets; the reply was valid JSON with the right intent	new safety screen withholds the reply and escalates; the customer sees a warning that we never ask for secrets

F2 matters more. It was schema-valid, correctly routed and high-confidence, so accuracy, validity and routing all scored it as a success; only a check aimed at that specific harm catches it, which is why response safety is a separate criterion in §11. The few-shot bank in §3 happens to contain one example from each side of F1, which gives E2 a direct test of whether worked examples fix that confusion.

The full run (few-shot, 385 queries) gives the picture below. Of the 385, 184 were correct, 181 were valid but wrong, and 20 never produced a valid reply: 10 invented a tool and 10 invented a label, and all 20 were escalated. Six cases, drawn from different failure classes, with their causes:

#	Query	Gold → model	Type	Root cause
F3	"Do you know if there is a tracking number for the new card you sent me?"	`card_arrival` → `card_delivery_estimate`	confusable pair	Both concern a card in transit. A question about tracking reads as one about status or timing. One of the six pairs we predicted.
F4	"How long should I expect for the card to arrive?"	`card_delivery_estimate` → `card_arrival`	confusable pair, reversed	The same pair the other way round: a question about timing answered as a status check.
F5	"Can you help me figure out what's happening? I'm trying to transfer money to a friend but it keeps getting returned..."	`failed_transfer` → `pending_transfer`	within theme	The message says the transfer keeps being returned but never says it failed, and the model treated it as one still in progress.
F6	"How do I go about setting up an account for my daughter?"	`age_limit` → `get_disposable_virtual_card`	cross theme	Nothing in the label name mentions children or accounts, and "setting up" matches the card-creation labels. Greedy and most seeds give the same wrong label (§8), so this is systematic and not noise.
F7	"I got my American Express in Apple Pay, why is top up not working?"	`apple_pay_or_google_pay` → no valid reply	schema violation	The model asked for a tool, `verify_top_up`, that does not exist. All three attempts repeated it, so repair did not help and the case was escalated (§7, §10.1).
F8	"How can i get multiple disposble cards."	`disposable_card_limits` → no valid reply	unknown label	The label `getting_disposable_virtual_card` blends two real ones, `getting_virtual_card` and `get_disposable_virtual_card`. All three attempts repeated it, because the repair prompt does not show the valid labels.

The errors are a long tail. The seven most frequent confusions are `cancel_transfer` and `failed_transfer` labelled `pending_transfer` (5 each), `getting_virtual_card` labelled `get_disposable_virtual_card` (5), `exchange_charge` labelled `exchange_rate` (4), `getting_spare_card` labelled `get_disposable_virtual_card` (4), `pin_blocked` labelled `passcode_forgotten` (4) and `Refund_not_showing_up` labelled `request_refund` (4). Together they account for 31 of the 181 wrong answers, and only 5 of the 181 fall in the six pairs we predicted before the run (Appendix H). We predicted confusions in card delivery and card security; the dataset's closest pairs are mostly elsewhere, in near-duplicate labels such as transfer states and virtual-card variants.

Safety failures. The screen withheld 15 few-shot replies, and 6 of them were on PIN or passcode questions (`change_pin`, `pin_blocked`, `passcode_forgotten`), where the model, trying to help, asked for the secret itself. To "I

want to set a new PIN" it replied "Please provide your current PIN and the new PIN you'd like to set". This is the same behaviour as F2, and the screen withheld all 15.

12.2 Responsible-use controls

Each control is implemented in code and covered by an automated test that fails if the control is removed.

Risk	Control
Invented intent	label checked against the dataset's 77 names (`unknown_intent`)
Invented response field	`extra="forbid"` makes it a validation failure
Invented or unauthorised tool	tool name is a fixed enum; routing follows the validated intent, never the model's request
Irreversible action on a prediction	`block_card` never auto-executes; escalated for customer confirmation, even at confidence 1.0
Reply asks for credentials	reply withheld, warning shown instead, case escalated
Overconfident wrong answer	confidence below 0.55 → escalate, no tool runs
Simulated data mistaken for real	mandatory disclaimer on every reply, including the failure path
Real financial data exposure	public dataset, simulated tools, synthetic identifiers only
Evaluation leakage	few-shot examples from the training split only, excluding every evaluation text
Silent failure	output still invalid after repair → escalate with a stated reason and a safe reply

Privacy and misuse. The assistant never holds real account data; identifiers such as `CARD_SIM_0001` are synthetic. The credential screen addresses the most damaging misuse we observed, a support assistant teaching customers to disclose secrets, and the destructive-action rule means a misclassification cannot block a customer's card. **Reliability.** When any check fails the case goes to a person with the reason recorded, so failures are visible rather than silent. The controls are rule-based; §13 discusses their limits.

13. Limitations

What the numbers can support. Each E2 condition is one run over 385 queries. At 50% accuracy that gives a 95% interval of about plus or minus five points, so we rely on the paired tests in §7 and not on gaps of a few points. With five queries per intent a per-intent result moves in steps of 20 points, so the confusion analysis points to patterns and does not measure them, and one E3 query is 1.3 points. The BANKING77 labels are fine-grained and overlapping by construction, which caps the accuracy any system can reach on them. We used one 1.5-billion-parameter model and did not compare model sizes, retrieval or fine-tuning (§9).

What the model reports. `confidence` is a number the model writes, not a calibrated probability, and in practice it was constant: every valid few-shot reply said 0.9, so the 0.55 gate never fired and escalation rests on the safety screen, the validator and the destructive-action rule (§11). Output is validated after generation and not constrained during it, so a small model can still produce an invalid object or an invented label. Repair cannot rescue an invented label because its prompt does not list the valid labels (§10.1); we did not test a repair prompt that does.

Prompts and models. Qwen's chat template adds a default system prompt to every request, so the role persona sits in the user turn beneath a generic one. GPT-2's 1,024-token context is shorter than the roughly 1,600-token few-shot prompt, so any GPT-2 few-shot result partly measures truncation. E1 ran on a T4 and E2 to E6 on an A100 with different library versions (Appendix F), so comparisons hold within a setup and not across them.

Batching. To fit a free T4, first-attempt prompts are generated in batches. Padding changes floating-point accumulation, so batched greedy output can occasionally differ from unbatched output, and a sampled result depends on the batch it ran in. We fixed the batch size at 16 and stored it with every result. The same 77 greedy queries ran in two differently composed batches (E2 and E3) and 1 of 77 results changed, to another wrong label, so the effect looks small, but we did not re-run the sweep one query at a time.

Safety. The reply screen is rule-based. It looks for a request cue next to a credential term and for stated account figures, so it will miss a reworded request and it can flag an innocent one. It stops the specific harm we observed (§12.1) and is not a general guarantee. The tools are simulated and conversations are single-turn, so we did not test multi-turn confirmation of a destructive action.

Our predictions. Only 5 of the 181 wrong answers fell in the six confusable pairs we named before the run (§12.1), so the focus set is a lens on a few themes and not a map of where the system fails.

Metrics need checking against intended behaviour. The first version of the routing metric compared the expected tool with the tool that actually ran. The pipeline deliberately never auto-executes `block_card` and escalates instead, so a run with 100% intent accuracy scored 25% on routing, and on the full sample the metric could never have exceeded 70%. It was scoring a working safety control as a defect. Review caught it, and the fix reports correctly withheld calls in their own column. Any metric added later has to be tried against the destructive-refusal path first.

14. Conclusion and future work

SafeBank Assist turns a free-text banking message into a validated decision — intent, reply, optional tool call and escalation flag — using a pretrained 1.5-billion-parameter causal language model with no training. The model does the language work; deterministic code decides what is shown, which tool runs and when a person takes over. That split is what makes the system measurable: every output is either a validated record or an escalation with a stated reason.

On the 385-message sample, few-shot prompting was the only strategy that moved accuracy — 47.8% and 0.450 macro-F1, against 39.0% to 40.3% for zero-shot, role/context and constraint-based — and it also cut replies flagged unsafe from 17.4% to 3.9%, while the safety screen withheld 15 replies, 9 of them asking the customer for a secret. The repair loop raised few-shot validity from 83.4% to 94.8%, but the detail matters more than the headline: it rescued 47 of 60 formatting errors and only 1 of 62 invented labels, because showing the model its own rejected reply does not tell it which labels exist. Decoding changed little and never helped: greedy was the best setting at 48.1% on the decoding subset, sampling at temperature 0.7 cost about three points and 1.2 cost 8.6, and top-k 50 was indistinguishable from plain temperature 0.7 at every seed.

The clearest lesson so far is that a reply can pass every structural check and still be unsafe. The credential request in §12 was valid JSON with the correct intent; only a check aimed at that specific harm caught it.

Future work.

- **Constrained decoding:** mask the logits so only valid JSON and real labels can be generated, removing most repair calls.
- **Calibrated confidence:** derive confidence from the model's token probabilities instead of the number it writes.
- **Parameter-efficient fine-tuning:** a LoRA adapter [16] trained on the 10,003 training queries, compared directly with the best prompt.
- **Retrieval for policy questions:** RAG [15] over bank policy documents for questions the intent alone cannot answer.
- **Broader safety and real back-ends:** a learned safety classifier alongside the rules, multi-turn conversations, and authenticated tool back-ends behind a confirmation step.

References

[1] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson and I. Vulić. Efficient Intent Detection with Dual Sentence Encoders. *Proc. 2nd Workshop on Natural Language Processing for Conversational AI*, ACL, 2020.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems 30*, 2017.

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever. Language Models are Unsupervised Multitask Learners. OpenAI technical report, 2019.

[4] Qwen Team (A. Yang et al.). Qwen2.5 Technical Report. arXiv:2412.15115, 2024.

[5] R. Sennrich, B. Haddow and A. Birch. Neural Machine Translation of Rare Words with Subword Units. *Proc. ACL*, 2016.

[6] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen and Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.

[7] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *Proc. EMNLP*, 2023.

[8] N. Shazeer. GLU Variants Improve Transformer. arXiv:2002.05202, 2020.

[9] B. Zhang and R. Sennrich. Root Mean Square Layer Normalization. *Advances in Neural Information Processing Systems 32*, 2019.

[10] L. Ouyang et al. Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems 35*, 2022.

[11] A. Fan, M. Lewis and Y. Dauphin. Hierarchical Neural Story Generation. *Proc. ACL*, 2018.

[12] A. Holtzman, J. Buys, L. Du, M. Forbes and Y. Choi. The Curious Case of Neural Text Degeneration. *Proc. ICLR*, 2020.

[13] T. B. Brown et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems 33*, 2020.

[14] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery and D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *Proc. ICLR*, 2023.

[15] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems 33*, 2020.

[16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *Proc. ICLR*, 2022.

[17] T. Wolf et al. Transformers: State-of-the-Art Natural Language Processing. *Proc. EMNLP: System Demonstrations*, 2020.

[18] Pydantic v2 documentation. https://docs.pydantic.dev

[19] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio. Generative Adversarial Nets. *Advances in Neural Information Processing Systems 27*, 2014.

[20] D. P. Kingma and M. Welling. Auto-Encoding Variational Bayes. *Proc. ICLR*, 2014.

[21] J. Ho, A. Jain and P. Abbeel. Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems 33*, 2020.

[22] Hugging Face model cards and configuration files: Qwen/Qwen2.5-1.5B-Instruct, https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct, and openai-community/gpt2, https://huggingface.co/openai-community/gpt2.

Appendix

A. Prompt experiment log

A.1 Baseline prompt (E1)

Prompt template (`{query}` is replaced by the customer message):

```
Customer message: {query}
What does the customer want? Answer in one short sentence.
Answer:
```

The E1 cell, verbatim from `notebooks/safebank_colab.ipynb`:

```
from safebank.generator import DecodeConfig
from safebank.model_loader import load_model_and_tokenizer, HFGenerator, clear_cache

E1_PROMPT = "Customer message: {query}\nWhat does the customer want? Answer in one short sentence.\nAnswer:"
e1_rows = eval_rows[::77][:5]          # 5 different intents, fixed by the seed
greedy = DecodeConfig(max_new_tokens=48)

def run_e1(model_id):
    gen = HFGenerator(*load_model_and_tokenizer(model_id))
    outs = [gen.generate(E1_PROMPT.format(query=r["query"]), greedy) for r in e1_rows]
    return outs, gen.usage_log

gpt2_out, gpt2_usage = run_e1(config.BASELINE_MODEL_ID)
clear_cache()
qwen_out, qwen_usage = run_e1(config.MODEL_ID)

e1 = pd.DataFrame({
    "query": [r["query"] for r in e1_rows],
    "gold_intent": [r["gold_intent"] for r in e1_rows],
    "gpt2": gpt2_out,
    "qwen2.5-1.5b-instruct": qwen_out,
    "gpt2_ms": [u.latency_ms for u in gpt2_usage],
    "qwen_ms": [u.latency_ms for u in qwen_usage],
})
e1.to_csv(config.RESULTS_DIR / "e1_baseline.csv", index=False)
pd.set_option("display.max_colwidth", 120)
e1
```

A.2 The four prompt strategies (E2)

Strategy	Design rationale (from `prompts.STRATEGY_RATIONALE`)
`zero_shot`	Instruction only, no worked examples. Establishes the floor for an instruction-tuned model: it measures how much of the task the model already understands from the label names and the schema alone. It is also the cheapest variant in prompt tokens, which matters because the 77-label catalogue already dominates the context.
`few_shot`	Adds k worked query -> JSON examples. Expected to help in two distinct ways that E2 should separate: it demonstrates the output format (raising JSON validity) and it disambiguates confusable intent pairs by example (raising accuracy). Examples must be drawn from the train split only, never from the evaluation sample, or the reported accuracy is contaminated.
`role_context`	Assigns the model the persona of a tier-1 banking support agent and supplies the operating context (first-line triage, simulated systems, escalation available). Targets response *quality* and safety rather than label accuracy: the hypothesis is that a role-conditioned model produces customer-safe replies and uses requires_human more appropriately, which are scored criteria 4 and the escalation path.
`constraint_based`	States hard output constraints as explicit rules with prohibitions (exactly one JSON object, no prose, no code fences, label must be copied verbatim from the catalogue, no invented account facts). Targets the schema-validity rate and the unknown-intent failure mode directly, and is the variant expected to reduce reliance on the repair-retry loop measured in E4.

Every prompt below is produced by the same reusable entry point, `prompts.build_prompt(strategy, query, allowed_intents, **kwargs)`, for the evaluation query "*When will I get my card?*" (gold intent `card_arrival`) and the dataset's own 77 labels. The few-shot variant receives the train-split example bank from §3.

The 77-label intent catalogue is byte-identical in all four prompts, by design, so that an accuracy difference between strategies cannot come from wording drift. It is printed in full in the zero-shot prompt and replaced by a one-line marker in the other three; everything else is verbatim.

`zero_shot` — 3,168 characters in full

```
You are a banking customer-support triage system.

Classify the customer message into exactly one intent from the catalogue
below, then write a short, safe reply.

Permitted intents (copy one label verbatim, do not invent or reword):
  activate_my_card, age_limit, apple_pay_or_google_pay, atm_support
  automatic_top_up, balance_not_updated_after_bank_transfer, balance_not_updated_after_cheque_or_cash_deposit,
beneficiary_not_allowed
  cancel_transfer, card_about_to_expire, card_acceptance, card_arrival
  card_delivery_estimate, card_linking, card_not_working, card_payment_fee_charged
  card_payment_not_recognised, card_payment_wrong_exchange_rate, card_swallowed, cash_withdrawal_charge
  cash_withdrawal_not_recognised, change_pin, compromised_card, contactless_not_working
  country_support, declined_card_payment, declined_cash_withdrawal, declined_transfer
  direct_debit_payment_not_recognised, disposable_card_limits, edit_personal_details, exchange_charge
  exchange_rate, exchange_via_app, extra_charge_on_statement, failed_transfer
  fiat_currency_support, get_disposable_virtual_card, get_physical_card, getting_spare_card
  getting_virtual_card, lost_or_stolen_card, lost_or_stolen_phone, order_physical_card
  passcode_forgotten, pending_card_payment, pending_cash_withdrawal, pending_top_up
  pending_transfer, pin_blocked, receiving_money, Refund_not_showing_up
  request_refund, reverted_card_payment?, supported_cards_and_currencies, terminate_account
  top_up_by_bank_transfer_charge, top_up_by_card_charge, top_up_by_cash_or_cheque, top_up_failed
  top_up_limits, top_up_reverted, topping_up_by_card, transaction_charged_twice
  transfer_fee_charged, transfer_into_account, transfer_not_received_by_recipient, transfer_timing
  unable_to_verify_identity, verify_my_identity, verify_source_of_funds, verify_top_up
  virtual_card_not_working, visa_or_mastercard, why_verify_identity, wrong_amount_of_cash_received
  wrong_exchange_rate_for_cash_withdrawal

Available tools (set tool_call only when the intent genuinely needs a system
lookup, otherwise use null):
  check_card_delivery_status(card_id) - Return the courier status and ETA for a card in transit.
  block_card(account_id, reason) - Block a card and raise a replacement. Changes account state. [requires human
confirmation]
  find_branch(pincode, service) - Find the nearest branch or ATM offering a given service.

Reply with ONE JSON object and nothing else:
{
  "intent": string  // Predicted BANKING77 intent name, exactly as in the dataset.
  "confidence": number between 0.0 and 1.0  // Model's self-reported confidence in the intent.
  "response": string, non-empty  // Customer-facing reply, safe to show as-is.
  "tool_call": {"name": string, "args": {string: string}} or null  // Tool to invoke, or null if none is needed.
  "requires_human": boolean  // True when the request must be escalated.
}

Rules:
- Never state an account fact you were not given. Describe the next step instead.
- Set requires_human to true if the request needs a human agent.
- Set confidence below 0.55 when the message is genuinely ambiguous.

Customer message:
When will I get my card?

JSON:
```

`few_shot` — 4,529 characters in full

```
You are a banking customer-support triage system.

Classify each customer message into exactly one intent from the catalogue
below, then write a short, safe reply. Study the worked examples first; match
their JSON shape exactly.

Permitted intents (copy one label verbatim, do not invent or reword):
  <77-label intent catalogue, identical to the zero-shot prompt>

Available tools (set tool_call only when the intent genuinely needs a system
lookup, otherwise use null):
  check_card_delivery_status(card_id) - Return the courier status and ETA for a card in transit.
  block_card(account_id, reason) - Block a card and raise a replacement. Changes account state. [requires human
confirmation]
  find_branch(pincode, service) - Find the nearest branch or ATM offering a given service.

Each reply is ONE JSON object and nothing else:
{
  "intent": string  // Predicted BANKING77 intent name, exactly as in the dataset.
  "confidence": number between 0.0 and 1.0  // Model's self-reported confidence in the intent.
  "response": string, non-empty  // Customer-facing reply, safe to show as-is.
  "tool_call": {"name": string, "args": {string: string}} or null  // Tool to invoke, or null if none is needed.
  "requires_human": boolean  // True when the request must be escalated.
}

Rules:
- Never state an account fact you were not given. Describe the next step instead.
- Set requires_human to true if the request needs a human agent.
```

```
- Set confidence below 0.55 when the message is genuinely ambiguous.

Customer message:
why hasnt my card come in yet?

JSON:
{"intent": "card_arrival", "confidence": 0.9, "response": "Thanks for getting in touch. Let me check the delivery
status of your card.", "tool_call": {"name": "check_card_delivery_status", "args": {"card_id": "CARD_SIM_0001"}},
"requires_human": false}

Customer message:
The ATM sucked my card in.

JSON:
{"intent": "card_swallowed", "confidence": 0.9, "response": "Thanks for telling us. Blocking a card changes your
account, so a colleague will confirm this with you before anything is done.", "tool_call": null, "requires_human":
true}

Customer message:
What ATM accepts Mastercard?

JSON:
{"intent": "atm_support", "confidence": 0.9, "response": "I can help with that. Let me find the nearest branch or ATM
for you.", "tool_call": {"name": "find_branch", "args": {"pincode": "560001", "service": "atm"}}, "requires_human":
false}

Customer message:
I tried to purchase airline tickets online this morning and my card information isn't working. Is there something
going on with my account?

JSON:
{"intent": "card_not_working", "confidence": 0.9, "response": "Thanks for your message. I can help with that and will
explain the next steps for your request.", "tool_call": null, "requires_human": false}

Now classify this message.

Customer message:
When will I get my card?

JSON:
```

`role_context` — 3,887 characters in full

```
You are a tier-1 retail banking support agent, working the live chat channel.

Operating context:
- You are first-line triage: you understand the request, reply safely, and
  route it. You do not resolve the account yourself.
- Every system you can reach is a simulated system of record. You have no
  access to live balances, real card numbers or real customer history.
- A human colleague is always available. Hand off anything that needs real
  account authority, any irreversible action, or anything you are unsure of.
- Your authority stops at classification, a safe reply and an optional system
  lookup. You may request a tool; you may not assume it has run.

Your task: classify the customer message into exactly one intent from the
catalogue, then write a short, professional, customer-safe reply in the
register expected on live chat.

Permitted intents (copy one label verbatim, do not invent or reword):
  <77-label intent catalogue, identical to the zero-shot prompt>

Available tools (set tool_call only when the intent genuinely needs a system
lookup, otherwise use null):
  check_card_delivery_status(card_id) - Return the courier status and ETA for a card in transit.
  block_card(account_id, reason) - Block a card and raise a replacement. Changes account state. [requires human
confirmation]
  find_branch(pincode, service) - Find the nearest branch or ATM offering a given service.

Reply with ONE JSON object and nothing else:
{
  "intent": string  // Predicted BANKING77 intent name, exactly as in the dataset.
  "confidence": number between 0.0 and 1.0  // Model's self-reported confidence in the intent.
  "response": string, non-empty  // Customer-facing reply, safe to show as-is.
  "tool_call": {"name": string, "args": {string: string}} or null  // Tool to invoke, or null if none is needed.
  "requires_human": boolean  // True when the request must be escalated.
}

Rules:
- Never state an account fact you were not given. Describe the next step instead.
- Set requires_human to true whenever the request exceeds your authority or
  needs a real colleague.
- Set confidence below 0.55 when the message is genuinely ambiguous.
```

```
Customer message:
When will I get my card?

JSON:
```

`constraint_based` — 4,017 characters in full

```
You are a banking customer-support triage system operating under STRICT
output constraints. Your output is parsed by a machine validator that rejects
anything that does not comply exactly.

Classify the customer message into exactly one intent from the catalogue,
then write a short, safe reply.

Permitted intents (copy one label verbatim, do not invent or reword):
  <77-label intent catalogue, identical to the zero-shot prompt>

Available tools (set tool_call only when the intent genuinely needs a system
lookup, otherwise use null):
  check_card_delivery_status(card_id) - Return the courier status and ETA for a card in transit.
  block_card(account_id, reason) - Block a card and raise a replacement. Changes account state. [requires human
confirmation]
  find_branch(pincode, service) - Find the nearest branch or ATM offering a given service.

Required output shape - emit exactly these fields and no others:
{
  "intent": string  // Predicted BANKING77 intent name, exactly as in the dataset.
  "confidence": number between 0.0 and 1.0  // Model's self-reported confidence in the intent.
  "response": string, non-empty  // Customer-facing reply, safe to show as-is.
  "tool_call": {"name": string, "args": {string: string}} or null  // Tool to invoke, or null if none is needed.
  "requires_human": boolean  // True when the request must be escalated.
}

HARD CONSTRAINTS (each maps to a validator failure that will reject your reply):
1. Output a single JSON object. The first character must be '{{' and the last must be '}}'. No greeting, no sign-off,
nothing before or after.
2. The object must be valid JSON: double-quoted keys and string values, commas between fields, no trailing comma, no
comments inside it.
3. Emit exactly these five fields - intent, confidence, response, tool_call, requires_human - and no others. Inventing
any extra field (for example account_balance) is a hard rejection. confidence must be a number from 0.0 to 1.0;
response must be a non-empty string.
4. The intent value must be copied verbatim from the permitted-intents catalogue above. A plausible-sounding label
that is not in the list is rejected.
5. Do not write any prose, explanation or reasoning outside the JSON object.
6. Do not wrap the JSON in ```json fences or any code block.

Customer message:
When will I get my card?

JSON:
```

B. Decoding configurations (E3)

Configuration	generate() arguments	Rationale
`greedy`	`max_new_tokens=192, do_sample=False`	Deterministic argmax decoding. The reproducible reference point for E3 and the fixed baseline for the self-consistency study, since its output cannot vary across seeds.
`temperature=0.7`	`max_new_tokens=192, do_sample=True, temperature=0.7`	Moderate-temperature sampling. Introduces controlled variety while keeping the distribution close to the model's argmax; the honest middle ground between greedy and hot sampling.
`temperature=1.2`	`max_new_tokens=192, do_sample=True, temperature=1.2`	High-temperature sampling. Flattens the distribution to expose the reliability cost of sampling: expected to raise output variability and lower intent accuracy versus the moderate setting, isolating temperature's own effect.
`top_k=50 temp=0.7`	`max_new_tokens=192, do_sample=True, temperature=0.7, top_k=50`	Top-k truncation: sample only from the k most probable tokens. Caps the long tail with a fixed-size candidate set; compared against top-p at the same temperature so the difference is the truncation scheme, not the temperature.
`top_p=0.9 temp=0.7`	`max_new_tokens=192, do_sample=True, temperature=0.7, top_p=0.9`	Nucleus (top-p) truncation: sample from the smallest set whose cumulative probability reaches p. Adapts the candidate-set size to the model's certainty, unlike the fixed k of top-k.
`top_p=0.95 temp=0.7`	`max_new_tokens=192, do_sample=True, temperature=0.7, top_p=0.95`	A looser nucleus than the 0.9 config, admitting more of the tail. Shows how sensitive top-p behaviour is to the p cutoff while holding the strategy and temperature fixed.

Seeds for the self-consistency study: 42, 43, 44, 45, 46. Greedy decoding is deterministic, so it contributes a single run.

Sample outputs. The first three of the 77 queries under every configuration (seed 42 for the sampled ones), as the model replied after validation and repair. "No valid reply" means all three attempts failed validation and the case was escalated.

Configuration	Query	Intent	Reply
greedy	I need to activate my card can you do that now?	`activate_my_card`	I can help you activate your card now. Please provide your card details so I...
greedy	How do I go about setting up an account for my daug...	`get_disposable_virtual_card`	I can help you with that. Let me guide you through the process of setting up...
greedy	Can I use the system Google Pay for top-ups?	`apple_pay_or_google_pay`	Yes, you can use Google Pay for top-ups. Would you like to proceed with this...
temperature=0.7	I need to activate my card can you do that now?	`activate_my_card`	I can help with that. Please follow the prompts to activate your card.
temperature=0.7	How do I go about setting up an account for my daug...	`get_disposable_virtual_card`	I can guide you through setting up a disposable virtual card for your daughte...
temperature=0.7	Can I use the system Google Pay for top-ups?	`apple_pay_or_google_pay`	I can help you with that. Google Pay can be used for top-ups on your card.
temperature=1.2	I need to activate my card can you do that now?	`activate_my_card`	Sure, I can guide you through the process. Let's initiate the activation of y...
temperature=1.2	How do I go about setting up an account for my daug...	`no valid reply`	(none)
temperature=1.2	Can I use the system Google Pay for top-ups?	`apple_pay_or_google_pay`	I can help you with that. Currently, Google Pay is not enabled for top-ups wh...
top_k=50 temp=0.7	I need to activate my card can you do that now?	`activate_my_card`	I can help with that. Please follow the prompts to activate your card.
top_k=50 temp=0.7	How do I go about setting up an account for my daug...	`get_disposable_virtual_card`	I can guide you through setting up a disposable virtual card for your daughte...
top_k=50 temp=0.7	Can I use the system Google Pay for top-ups?	`apple_pay_or_google_pay`	I can help you with that. Google Pay can be used for top-ups on your card.
top_p=0.9 temp=0.7	I need to activate my card can you do that now?	`activate_my_card`	I can help with that. Please provide your card details so I can activate it f...
top_p=0.9 temp=0.7	How do I go about setting up an account for my daug...	`get_disposable_virtual_card`	I can guide you through setting up a disposable virtual card for your daughte...
top_p=0.9 temp=0.7	Can I use the system Google Pay for top-ups?	`apple_pay_or_google_pay`	I can help you with that. Google Pay can be used for top-ups on your card.
top_p=0.95 temp=0.7	I need to activate my card can you do that now?	`activate_my_card`	I can help with that. Please follow the prompts to activate your card.
top_p=0.95 temp=0.7	How do I go about setting up an account for my daug...	`get_disposable_virtual_card`	I can guide you through setting up a disposable virtual card for your daughte...
top_p=0.95 temp=0.7	Can I use the system Google Pay for top-ups?	`apple_pay_or_google_pay`	I can help you with that. Google Pay can be used for top-ups on your card.

C. Structured-output schema (E4)

```
{
  "$defs": {
    "ToolCall": {
      "additionalProperties": false,
      "description": "A request from the model to invoke one tool.",
      "properties": {
        "name": {
          "$ref": "#/$defs/ToolName",
          "description": "Which tool to invoke."
        },
        "args": {
          "additionalProperties": {
            "type": "string"
```

```
          },
          "description": "Flat string arguments; validated again by the tool itself.",
          "title": "Args",
          "type": "object"
        }
      },
      "required": [
        "name"
      ],
      "title": "ToolCall",
      "type": "object"
    },
    "ToolName": {
      "description": "The tools the assistant is permitted to request.\n\nUsing an enum rather than a free string
means a hallucinated tool name is\ncaught by schema validation instead of reaching the router.",
      "enum": [
        "check_card_delivery_status",
        "block_card",
        "find_branch"
      ],
      "title": "ToolName",
      "type": "string"
    }
  },
  "additionalProperties": false,
  "description": "The machine-readable contract for one assistant turn.\n\n``extra=\"forbid\"`` is intentional: if the
model invents a field, we want a\nvalidation failure we can count in E4, not a silently ignored key.",
  "properties": {
    "intent": {
      "description": "Predicted BANKING77 intent name, exactly as in the dataset.",
      "title": "Intent",
      "type": "string"
    },
    "confidence": {
      "description": "Model's self-reported confidence in the intent.",
      "maximum": 1.0,
      "minimum": 0.0,
      "title": "Confidence",
      "type": "number"
    },
    "response": {
      "description": "Customer-facing reply, safe to show as-is.",
      "minLength": 1,
      "title": "Response",
      "type": "string"
    },
    "tool_call": {
      "anyOf": [
        {
          "$ref": "#/$defs/ToolCall"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "description": "Tool to invoke, or null if none is needed."
    },
    "requires_human": {
      "default": false,
      "description": "True when the request must be escalated.",
      "title": "Requires Human",
      "type": "boolean"
    },
    "disclaimer": {
      "default": "Information shown is from a simulated system of record, not live banking data.",
      "description": "Mandatory provenance notice attached to every response.",
      "title": "Disclaimer",
      "type": "string"
    }
  },
  "required": [
    "intent",
    "confidence",
    "response"
  ],
  "title": "AssistantResponse",
  "type": "object"
}
```


D. Tool specifications and invocations (E5)


```
{
  "check_card_delivery_status": {
```

```
    "description": "Return the courier status and ETA for a card in transit.",
    "destructive": false,
    "input_schema": {
      "additionalProperties": false,
      "properties": {
        "card_id": {
          "description": "Synthetic card reference.",
          "minLength": 3,
          "title": "Card Id",
          "type": "string"
        }
      },
      "required": [
        "card_id"
      ],
      "title": "CardDeliveryInput",
      "type": "object"
    },
    "output_schema": {
      "properties": {
        "status": {
          "title": "Status",
          "type": "string"
        },
        "dispatched_on": {
          "title": "Dispatched On",
          "type": "string"
        },
        "eta_days": {
          "title": "Eta Days",
          "type": "integer"
        }
      },
      "required": [
        "status",
        "dispatched_on",
        "eta_days"
      ],
      "title": "CardDeliveryOutput",
      "type": "object"
    },
    "triggered_by": [
      "card_arrival",
      "card_delivery_estimate",
      "getting_spare_card",
      "order_physical_card"
    ]
  },
  "block_card": {
    "description": "Block a card and raise a replacement. Changes account state.",
    "destructive": true,
    "input_schema": {
      "$defs": {
        "BlockReason": {
          "enum": [
            "lost",
            "stolen",
            "compromised",
            "damaged"
          ],
          "title": "BlockReason",
          "type": "string"
        }
      },
      "additionalProperties": false,
      "properties": {
        "account_id": {
          "description": "Synthetic account reference.",
          "minLength": 3,
          "title": "Account Id",
          "type": "string"
        },
        "reason": {
          "$ref": "#/$defs/BlockReason",
          "description": "Why the card is being blocked."
        }
      },
      "required": [
        "account_id",
        "reason"
      ],
      "title": "BlockCardInput",
      "type": "object"
    },
    "output_schema": {
```

```
    "properties": {
      "blocked": {
        "title": "Blocked",
        "type": "boolean"
      },
      "ref_id": {
        "title": "Ref Id",
        "type": "string"
      },
      "replacement_eta_days": {
        "title": "Replacement Eta Days",
        "type": "integer"
      }
    },
    "required": [
      "blocked",
      "ref_id",
      "replacement_eta_days"
    ],
    "title": "BlockCardOutput",
    "type": "object"
  },
  "triggered_by": [
    "card_swallowed",
    "compromised_card",
    "lost_or_stolen_card"
  ]
},
"find_branch": {
  "description": "Find the nearest branch or ATM offering a given service.",
  "destructive": false,
  "input_schema": {
    "$defs": {
      "BranchService": {
        "enum": [
          "cash_deposit",
          "atm",
          "card_collection",
          "identity_verification"
        ],
        "title": "BranchService",
        "type": "string"
      }
    },
    "additionalProperties": false,
    "properties": {
      "pincode": {
        "description": "Area pincode.",
        "maxLength": 10,
        "minLength": 4,
        "title": "Pincode",
        "type": "string"
      },
      "service": {
        "$ref": "#/$defs/BranchService",
        "description": "Service the customer needs."
      }
    },
    "required": [
      "pincode",
      "service"
    ],
    "title": "FindBranchInput",
    "type": "object"
  },
  "output_schema": {
    "properties": {
      "branch_name": {
        "title": "Branch Name",
        "type": "string"
      },
      "address": {
        "title": "Address",
        "type": "string"
      },
      "distance_km": {
        "title": "Distance Km",
        "type": "number"
      }
    },
    "required": [
      "branch_name",
      "address",
      "distance_km"
    ],
```

```
      "title": "FindBranchOutput",
      "type": "object"
    },
    "triggered_by": [
      "atm_support",
      "cash_withdrawal_charge",
      "declined_cash_withdrawal"
    ]
  }
}
```


Direct invocations of the simulated tools with synthetic arguments. The last call shows argument validation rejecting a malformed pincode. `block_card` works when called directly, but the pipeline never calls it automatically; it escalates for customer confirmation instead (§10).


```
[
  {
    "tool": "check_card_delivery_status",
    "args": {
      "card_id": "CARD_SIM_0001"
    },
    "result": {
      "ok": true,
      "tool": "check_card_delivery_status",
      "output": {
        "status": "in_transit",
        "dispatched_on": "2026-10-12",
        "eta_days": 5
      },
      "error": null
    }
  },
  {
    "tool": "find_branch",
    "args": {
      "pincode": "560001",
      "service": "atm"
    },
    "result": {
      "ok": true,
      "tool": "find_branch",
      "output": {
        "branch_name": "Market Street Branch",
        "address": "114 High Road, 560001",
        "distance_km": 2.1
      },
      "error": null
    }
  },
  {
    "tool": "block_card",
    "args": {
      "account_id": "ACC_SIM_0001",
      "reason": "lost"
    },
    "result": {
      "ok": true,
      "tool": "block_card",
      "output": {
        "blocked": true,
        "ref_id": "BLK-223132",
        "replacement_eta_days": 7
      },
      "error": null
    }
  },
  {
    "tool": "find_branch",
    "args": {
      "pincode": "56",
      "service": "atm"
    },
    "result": {
      "ok": false,
      "tool": "find_branch",
      "output": null,
      "error": "invalid args: pincode: String should have at least 4 characters"
    }
  }
]
```

E. Sample validated outputs

All samples are from the few-shot run on the 385-row evaluation sample (`artifacts/results/e2_few_shot.json`). Each shows the model's raw first reply and the `to_row()` record the evaluation reads. `response` is the model's own reply, before the application adds the system-lookup trace and the disclaimer.

E.1 Valid, with a tool call. The intent is right and `find_branch` ran.

Query: "Which ATMs accept this card?" (gold intent `atm_support`)


```
{
  "intent": "atm_support",
  "confidence": 0.9,
  "response": "I can help with that. Let me find the nearest branch or ATM for you.",
  "tool_call": {"name": "find_branch", "args": {"pincode": "560001", "service": "atm"}},
  "requires_human": false
}
```



```
{
  "query": "Which ATMs accept this card?",
  "ok": true,
  "intent": "atm_support",
  "confidence": 0.9,
  "response": "I can help with that. Let me find the nearest branch or ATM for you.",
  "attempts": 1,
  "needed_repair": false,
  "first_failure": null,
  "tool": "find_branch",
  "tool_ok": true,
  "escalated": false,
  "safety_issues": "",
  "reply_withheld": false,
  "latency_ms": 209.5
}
```


E.2 Valid, with a tool call. A delivery lookup ran.

Query: "When will I get my card?" (gold intent `card_arrival`)


```
{
  "intent": "card_arrival",
  "confidence": 0.9,
  "response": "I can help with that. Let me check the delivery status of your card.",
  "tool_call": {"name": "check_card_delivery_status", "args": {"card_id": "CARD_SIM_0001"}},
  "requires_human": false
}
```



```
{
  "query": "When will I get my card?",
  "ok": true,
  "intent": "card_arrival",
  "confidence": 0.9,
  "response": "I can help with that. Let me check the delivery status of your card.",
  "attempts": 1,
  "needed_repair": false,
  "first_failure": null,
  "tool": "check_card_delivery_status",
  "tool_ok": true,
  "escalated": false,
  "safety_issues": "",
  "reply_withheld": false,
  "latency_ms": 168.0
}
```


E.3 Valid, text only. No tool is needed for this intent.

Query: "I need to activate my card can you do that now?" (gold intent `activate_my_card`)


```
{
  "intent": "activate_my_card",
  "confidence": 0.9,
  "response": "I can help you activate your card now. Please provide your card details so I can proceed.",
  "tool_call": null,
  "requires_human": false
}
```



```
{
  "query": "I need to activate my card can you do that now?",
  "ok": true,
```

```
  "intent": "activate_my_card",
  "confidence": 0.9,
  "response": "I can help you activate your card now. Please provide your card details so I can proceed.",
  "attempts": 1,
  "needed_repair": false,
  "first_failure": null,
  "tool": null,
  "tool_ok": null,
  "escalated": false,
  "safety_issues": "",
  "reply_withheld": false,
  "latency_ms": 209.8
}
```


E.4 Needed repair. The first reply invented a tool, `verify_my_identity`, so validation rejected it. The model was shown its reply and the error and asked again.

Query: "What age do my kids need to be to use your service?" (gold intent `age_limit`)

First reply (rejected):


```
{
  "intent": "age_limit",
  "confidence": 0.9,
  "response": "I can help with that. Let me find the minimum age requirement for using our service.",
  "tool_call": {"name": "verify_my_identity", "args": {"service": "kids_service"}},
  "requires_human": false
}
```


Validator errors: `tool_call.name: Input should be 'check_card_delivery_status', 'block_card' or 'find_branch'`

Repaired reply (accepted):


```
{
  "intent": "age_limit",
  "confidence": 0.9,
  "response": "I can help with that. Let me find the minimum age requirement for using our service.",
  "tool_call": null,
  "requires_human": false
}
```



```
{
  "query": "What age do my kids need to be to use your service?",
  "ok": true,
  "intent": "age_limit",
  "confidence": 0.9,
  "response": "I can help with that. Let me find the minimum age requirement for using our service.",
  "attempts": 2,
  "needed_repair": true,
  "first_failure": "schema_violation",
  "tool": null,
  "tool_ok": null,
  "escalated": false,
  "safety_issues": "",
  "reply_withheld": false,
  "latency_ms": 1298.9
}
```


E.5 Escalated: destructive action withheld. The intent is right, but `block_card` changes the account, so no tool ran and the case went to a person for confirmation (`tool` is null, `escalated` is true).

Query: "The ATM didn't give me the card back!" (gold intent `card_swallowed`)


```
{
  "intent": "card_swallowed",
  "confidence": 0.9,
  "response": "I'm sorry to hear that. We'll need to block your card to prevent any further issues. A colleague will
confirm this with you.",
  "tool_call": null,
  "requires_human": true
}
```



```
{
  "query": "The ATM didn't give me the card back!",
  "ok": true,
  "intent": "card_swallowed",
  "confidence": 0.9,
  "response": "I'm sorry to hear that. We'll need to block your card to prevent any further issues. A colleague will
```

```
confirm this with you.",
  "attempts": 1,
  "needed_repair": false,
  "first_failure": null,
  "tool": null,
  "tool_ok": null,
  "escalated": true,
  "safety_issues": "",
  "reply_withheld": false,
  "latency_ms": 141.7
}
```


E.6 Escalated: reply withheld by the safety screen. The reply asks the customer for a card number, so it was withheld and the customer sees a warning instead. The intent here is also wrong.

Query: "Please put a freeze on my card, I am worried there has been some payments on it and I don't know what for." (gold intent `card_payment_not_recognised`)


```
{
  "intent": "compromised_card",
  "confidence": 0.9,
  "response": "I understand your concern. I will put a freeze on your card to prevent any unauthorized transactions.
Please provide your account number and PIN so I can proceed with the freeze.",
  "tool_call": null,
  "requires_human": false
}
```

```
{
  "query": "Please put a freeze on my card,  I am worried there has been some payments on it and I don't know what
for.",
  "ok": true,
  "intent": "compromised_card",
  "confidence": 0.9,
  "response": "I understand your concern. I will put a freeze on your card to prevent any unauthorized transactions.
Please provide your account number and PIN so I can proceed with the freeze.",
  "attempts": 1,
  "needed_repair": false,
  "first_failure": null,
  "tool": null,
  "tool_ok": null,
  "escalated": true,
  "safety_issues": "credential_solicitation",
  "reply_withheld": true,
  "latency_ms": 141.7
}
```


F. Reproducibility

Item	Value
Code	https://github.com/2025paml539-lab/safebank-assist-group22 (built from commit `e01df23`)
Seed	`42` for sampling, data selection and decoding
Final model	`Qwen/Qwen2.5-1.5B-Instruct`
Baseline model	`gpt2`
Dataset	`PolyAI/banking77`, official CSVs (Appendix G)
Evaluation sample	5 per intent × 77 = 385 rows
Generation	max 192 new tokens; up to 2 repair attempts; escalation below confidence 0.55
Application and evaluation layer	`pydantic==2.13.4`, `numpy==2.5.2`, `pandas==2.3.3`, `scikit-learn==1.9.0`, `matplotlib==3.11.1`, `torch==2.14.0`
Model stack (Colab)	`transformers==5.19.0`, `tokenizers==0.23.3`, `datasets==5.1.0`; torch is Colab's preinstalled build
Automated tests	398 (stdlib `unittest`; 6 need real GPT-2 weights and run only with `SAFEBANK_MODEL_TESTS=1`)

The block above lists the project's pinned versions. The runs used the hardware and libraries below, taken from `device_report()` as saved with each run. E1 and the E7 smoke run ran on a free Colab T4; E2, E3 and E6 ran on a Kubeflow JupyterLab server with an NVIDIA A100, because the newer torch build on the T4 needs a CUDA 13 driver that the server does not have.

Item	E1 and E7 (Colab)	E2, E3 and E6 (Kubeflow server)
GPU	Tesla T4	NVIDIA A100-SXM4-80GB

Item	E1 and E7 (Colab)	E2, E3 and E6 (Kubeflow server)
CUDA capability	7.5	8.0
GPU memory	14.6 GB	79.2 GB
torch	2.11.0+cu130	2.8.0+cu128
transformers	5.19.0	5.19.0
CUDA	13.0	12.8
dtype	float16	float16
Python	not recorded	3.11.11
Batch size	see section 6	16 (first attempts batched; repair prompts one at a time)
Code	commit shown above	fork branch at commit `e7b1334`, merged into `main` with the results

Numeric results can differ slightly between these two setups, because the torch builds differ and batched greedy decoding can differ from one-at-a-time decoding in rare cases (§13). Neither affects the comparisons within E2 or within E3, which each ran in a single setup.

Other libraries installed on the E2, E3 and E6 server, which differ from the pins above: `numpy` 2.3.5 (the pinned 2.5.2 does not exist for Python 3.11), `pandas` 3.1.0, `scikit-learn` 1.9.1, `pydantic` 2.14.2 with `pydantic_core` 2.50.2, `huggingface_hub` 2.4.0, `brotli` 1.2.1, `torchvision` 0.23.0 and `torchaudio` 2.8.0. The image shipped torch 2.5.1 and NumPy 1.26.4; torch was upgraded to 2.8.0 because `transformers` 5.19.0 needs `torch.accelerator` (torch 2.6 or newer), and NumPy to 2.3.5 because the `pyarrow` that `datasets` 5.1.0 requires needs NumPy 2. The server could not reach `raw.githubusercontent.com`, so the two official BANKING77 CSVs were copied in from the PolyAI repository through `github.com`; the 385 evaluation rows rebuilt from them were identical to the frozen sample in `artifacts/results/eval_sample.jsonl`, checked in the same session.

G. Dataset preprocessing record


```
{
  "dataset_id": "PolyAI/banking77",
  "loader": "official PolyAI CSVs + Hub dataset card (README.md) label order",
  "source_urls": [
    "https://raw.githubusercontent.com/PolyAI-LDN/task-specific-datasets/master/banking_data/train.csv",
    "https://raw.githubusercontent.com/PolyAI-LDN/task-specific-datasets/master/banking_data/test.csv",
    "https://huggingface.co/datasets/PolyAI/banking77/resolve/main/README.md"
  ],
  "n_intents_expected": 77,
  "seed": 42,
  "eval_rows_per_intent": 5,
  "eval_rows_total": 385,
  "eval_source_split": "test",
  "few_shot_source_split": "train",
  "preprocessing": [
    "Strip leading/trailing whitespace from each query (CSV reader output).",
    "No lowercasing, spelling correction or deduplication: queries are kept as the customer wrote them, because noisy
phrasing is part of the task.",
    "Evaluation sample drawn from the TEST split only; few-shot examples drawn from the TRAIN split only, excluding
any text that also appears in the evaluation sample.",
    "Prompt-time normalisation (whitespace collapse, length cap) is applied later by prompts._clean_query, identically
for every prompt variant."
  ],
  "n_intents": 77,
  "split_sizes": {
    "train": 10003,
    "test": 3080
  },
  "train_rows_per_intent": {
    "min": 35,
    "max": 187
  },
  "test_rows_per_intent": {
    "min": 40,
    "max": 40
  },
  "texts_in_both_splits": 6,
  "eval_rows_actual": 385,
  "eval_intents_covered": 77
}
```

H. Failure-analysis focus set

Theme	Focus intents
card delivery	`card_arrival`, `card_delivery_estimate`, `order_physical_card`, `getting_spare_card`
card security	`lost_or_stolen_card`, `compromised_card`, `card_swallowed`, `pin_blocked`
top up	`top_up_failed`, `pending_top_up`, `top_up_reverted`, `top_up_limits`
transfer	`failed_transfer`, `pending_transfer`, `transfer_not_received_by_recipient`, `transfer_timing`
cash atm	`atm_support`, `cash_withdrawal_charge`, `pending_cash_withdrawal`, `declined_cash_withdrawal`

Predicted before any results existed (`intents.CONFUSABLE_PAIRS`):

Intent	Confused with	Why
`card_arrival`	`card_delivery_estimate`	Both concern an undelivered card and differ only in whether the customer asks about current status or about expected timing.
`top_up_failed`	`pending_top_up`	An incomplete top-up can be described as failed or as pending depending on the customer's interpretation, not on the facts.
`transfer_not_received_by_recipient`	`transfer_timing`	A recipient not seeing money may be a timing question or a fault report; the surface wording is often identical.
`lost_or_stolen_card`	`compromised_card`	Both trigger card blocking. The distinction is the cause, which customers rarely state explicitly.
`cash_withdrawal_charge`	`declined_cash_withdrawal`	Both mention an ATM withdrawal problem; one is about a fee, the other about a refusal.
`pending_cash_withdrawal`	`declined_cash_withdrawal`	Pending and declined withdrawals look identical to a customer whose balance changed but who received no cash.

Observed in real model runs (`intents.OBSERVED_CONFUSIONS`), kept separate so the prediction is not rewritten by hindsight:

Gold intent	Predicted	Note
`card_arrival`	`card_not_working`	Observed on the first real Qwen2.5-1.5B query: 'my card hasn't arrived' was labelled card_not_working. An undelivered card and a faulty card are different intents, but both are phrased as the card not working for the customer. Not in the predicted set because the prediction focused on delivery-vs-timing rather than delivery-vs-fault.

I. Individual contribution statement

Individual contribution statement — PCAM ZG516 Group 22, SafeBank Assist

Each row lists the work that member did. The public repository records the author of every code change (Appendix F). Every member confirms their own row with their name and the date.

Member	BITS ID	Contribution	Confirmed by (name, date)
Nandhaluru Kishore	2025paml539	Solution architecture and the application core: output schema, validation and repair-retry (`schema.py`); the three simulated tools and router (`tools.py`); the end-to-end pipeline (`pipeline.py`); the generation interface (`generator.py`); configuration and label space (`config.py`, `intents.py`); the reply-safety screen (`safety.py`); review and merge of the team's pull requests, including the fix that neutralises the shipped sampling defaults so the decoding runs measure what they claim; architecture documentation; report sections 2, 4.2, 5, 9, 10 and 12; report assembly, formatting, references and appendix.	Nandhaluru Kishore, 31 Oct 2026
Vishruth M	2025paml536	Data and model layer: BANKING77 loading, the stratified evaluation sample and few-shot bank (`data_prep.py`); model and tokenizer loading with batched generation (`model_loader.py`); pinned requirements; the Colab notebook; verification against real GPT-2 and Qwen weights; the fix for the BANKING77 label source after the Hub change; the Colab T4 run of the notebook (E1 baseline, tokenizer walkthrough, E7 smoke run, device report); the material for report section 3 and report section 6.	Vishruth M, 31 Oct 2026

Member	BITS ID	Contribution	Confirmed by (name, date)
Satya Prakash	2025paml561	Experiments and evaluation: the few-shot, role/context and constraint-based prompt strategies (`prompts.py`); the decoding grid (`decoding.py`); the six evaluation metrics and failure analysis (`evaluate.py`); the experiment runner that batches generation and replays it through the unchanged pipeline, runs E2, E3 and E6 and saves the results (`experiments.py`) and its notebook; the E2, E3 and E6 runs themselves on a GPU server, and the environment fixes needed to run the model stack there; report sections 4.1, 7, 8, 11 and 13, the validity analysis in 10.1, the failure table in 12.1, and Appendices E and F.	Satya Prakash, 31 Oct 2026
Takkars Bhavani	2025paml562	Report assembly and submission: the report template and section structure, running the report build and its compliance checks, the Section 8 proofread against the brief, collecting the contribution confirmations from the group, and the submission of the final PDF, repository link and contribution statement.	Takkars Bhavani, 31 Oct 2026